

UNIVERSIDADE POSITIVO
CURSO DE DESENVOLVIMENTO DE SISTEMAS

HELPDESK PRO
SISTEMA DE GESTÃO DE CHAMADOS CORPORATIVOS

Emerson Junior
Luiz Gustavo Amaral
Maria Vitória
Samanta Andrade
Taynara Piloneto Mafra

Curitiba
2025

Emerson Junior
Luiz Gustavo Amaral
Maria Vitória
Samanta Andrade
Taynara Piloneto Mafra

HELPDESK PRO

SISTEMA DE GESTÃO DE CHAMADOS CORPORATIVOS

Trabalho acadêmico apresentado à disciplina de Desenvolvimento de Sistemas do curso de Desenvolvimento de Sistemas da Universidade Positivo, como requisito parcial para avaliação, consistindo no desenvolvimento de um sistema web de gestão de chamados em arquitetura MVC.

Curitiba
2025

SUMÁRIO

1 INTRODUÇÃO.....	4
1.1 Objetivos.....	4
1.1.1 Objetivo geral.....	4
1.1.2 Objetivos específicos.....	4
1.2 Justificativa.....	5
2 FUNDAMENTAÇÃO TEÓRICA.....	5
2.1 Linguagem PHP.....	5
2.2 Padrão arquitetural MVC.....	5
2.3 Banco de dados MySQL e PDO.....	5
2.4 Tecnologias de interface.....	6
3 MATERIAIS E MÉTODOS.....	6
4 MODELAGEM DO SISTEMA.....	6
4.1 Diagrama de casos de uso.....	6
4.2 Modelo entidade-relacionamento.....	7
4.3 Diagrama de componentes.....	8
4.4 Diagrama de classes.....	9
5 ARQUITETURA E IMPLEMENTAÇÃO.....	10
5.1 Camada de modelo.....	10
5.2 Camada de controle.....	10
5.3 Camada de visão.....	10
5.4 Segurança.....	10
5.5 API REST.....	11
6 FUNCIONALIDADES.....	11
7 CONSIDERAÇÕES FINAIS.....	11
REFERÊNCIAS.....	13

1 INTRODUÇÃO

Em ambientes corporativos, o atendimento a solicitações de suporte técnico — falhas de hardware, problemas em sistemas, indisponibilidade de rede e pedidos de acesso — costuma ocorrer por canais informais como e-mail, telefone ou mensagens. Essa dispersão dificulta o acompanhamento das demandas, compromete o cumprimento de prazos e impede a geração de indicadores confiáveis sobre o desempenho da equipe de suporte.

Diante desse cenário, o presente trabalho apresenta o HelpDesk Pro, um sistema web para gestão de chamados corporativos desenvolvido em linguagem PHP, com banco de dados MySQL e arquitetura Model-View-Controller (MVC). O sistema centraliza a abertura, o atendimento e o encerramento de chamados, organiza as demandas por categoria e prioridade, controla prazos por meio de Acordos de Nível de Serviço (SLA) e oferece um painel gerencial com indicadores visuais.

O sistema contempla três perfis de usuário — cliente, atendente e administrador —, cada qual com permissões específicas, e disponibiliza recursos como quadro Kanban, comentários públicos e internos, anexos, notificações, auditoria de ações e uma interface de programação de aplicações (API) no formato REST. Este documento descreve a modelagem, a arquitetura e as funcionalidades implementadas.

1.1 Objetivos

1.1.1 Objetivo geral

Desenvolver um sistema web de gestão de chamados corporativos que centralize as solicitações de suporte, controle prazos de atendimento e forneça indicadores gerenciais, aplicando o padrão arquitetural MVC em PHP.

1.1.2 Objetivos específicos

- Modelar o domínio do problema por meio de diagramas de casos de uso, entidade-relacionamento, componentes e classes;
- Implementar autenticação de usuários com controle de sessão e diferenciação de perfis de acesso;
- Construir o cadastro e o acompanhamento de chamados com categorias, prioridades e controle de SLA;
- Disponibilizar um painel gerencial com gráficos e um quadro Kanban para acompanhamento visual do fluxo de trabalho;
- Adotar boas práticas de segurança, como consultas preparadas, proteção contra CSRF e armazenamento de senhas com hash;
- Expor uma API REST para consulta de chamados, estatísticas e notificações.

1.2 Justificativa

A adoção de uma ferramenta de help desk traz padronização ao atendimento, rastreabilidade das solicitações e transparência sobre os prazos acordados. O registro estruturado das demandas permite identificar gargalos, mensurar a produtividade da equipe e priorizar incidentes críticos. Além do valor prático, o projeto constitui um exercício acadêmico relevante, pois integra conceitos de modelagem de sistemas, programação orientada a objetos, persistência de dados e segurança de aplicações web em um produto funcional.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Linguagem PHP

O PHP (Hypertext Preprocessor) é uma linguagem de programação de código aberto, de execução no lado servidor, amplamente utilizada no desenvolvimento de aplicações web dinâmicas. O projeto utiliza a versão 8.1 ou superior, que oferece recursos modernos como tipagem de propriedades, argumentos nomeados, expressões match e funções de seta (arrow functions), empregados ao longo do código para tornar as implementações mais concisas e seguras.

2.2 Padrão arquitetural MVC

O padrão Model-View-Controller organiza a aplicação em três camadas com responsabilidades distintas. A camada de modelo (Model) concentra as regras de negócio e o acesso aos dados; a camada de visão (View) é responsável pela apresentação das informações ao usuário; e a camada de controle (Controller) coordena o fluxo da aplicação, recebendo as requisições, acionando os modelos e selecionando as visões a serem exibidas. Essa separação favorece a manutenção, o reaproveitamento de código e o trabalho colaborativo, uma vez que cada camada pode evoluir de forma relativamente independente.

2.3 Banco de dados MySQL e PDO

O MySQL é um sistema gerenciador de banco de dados relacional utilizado para a persistência das informações. O acesso ao banco é realizado por meio da extensão PDO (PHP Data Objects), que oferece uma interface uniforme de conexão e, sobretudo, o uso de consultas preparadas (prepared statements). As consultas preparadas separam o comando SQL dos dados informados pelo usuário, constituindo a principal defesa contra ataques de injeção de SQL. No sistema, a conexão é encapsulada em uma classe que adota o padrão de projeto Singleton, garantindo uma única instância de conexão durante a requisição.

2.4 Tecnologias de interface

A interface emprega o framework Bootstrap 5 para a estruturação responsiva das páginas e o conjunto de ícones Bootstrap Icons. Os gráficos do painel gerencial são renderizados com a biblioteca Chart.js, enquanto o comportamento de arrastar e soltar (drag-and-drop) do quadro Kanban utiliza a biblioteca SortableJS.

3 MATERIAIS E MÉTODOS

O desenvolvimento adotou uma abordagem incremental, partindo da modelagem do sistema para, em seguida, implementar as camadas da arquitetura MVC. O ambiente de desenvolvimento utilizou o pacote XAMPP, que integra o servidor Apache e o banco de dados MySQL. O Quadro 1 resume as principais tecnologias e ferramentas empregadas.

Quadro 1 – Tecnologias utilizadas

- Linguagem de programação: PHP 8.1+;
- Banco de dados: MySQL (acesso via PDO com consultas preparadas);
- Servidor de aplicação: Apache (ambiente XAMPP);
- Front-end: HTML5, CSS3, Bootstrap 5, Bootstrap Icons;
- Bibliotecas JavaScript: Chart.js (gráficos) e SortableJS (Kanban);
- Arquitetura: MVC com roteamento por front controller (index.php).

O sistema é composto por cerca de 42 arquivos PHP, totalizando aproximadamente 4.500 linhas de código, distribuídos entre as camadas de modelo, controle e visão.

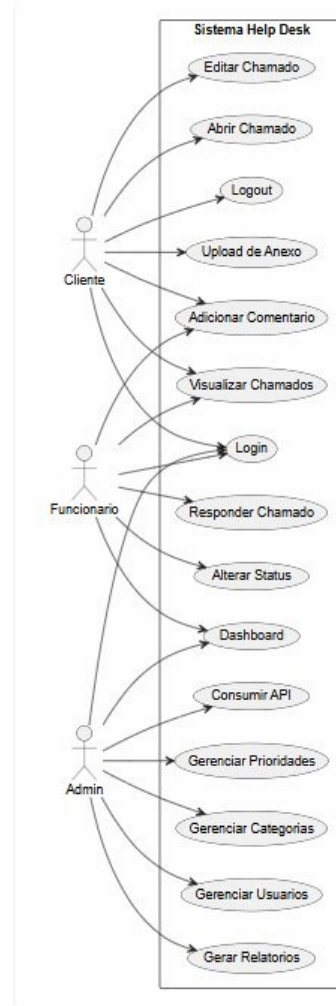
4 MODELAGEM DO SISTEMA

Esta seção apresenta a modelagem do HelpDesk Pro por meio de quatro diagramas: o diagrama de casos de uso, o modelo entidade-relacionamento, o diagrama de componentes e o diagrama de classes.

4.1 Diagrama de casos de uso

O diagrama de casos de uso, apresentado na Figura 1, descreve as interações entre os atores e o sistema. São identificados três atores: o Cliente, que abre, edita e visualiza seus chamados, adiciona comentários e anexos; o Funcionário (atendente), que responde chamados, altera status e acessa o painel; e o Administrador, que gerencia usuários, categorias e prioridades, gera relatórios e consome a API. As funcionalidades de Login e Logout são comuns aos três perfis.

Figura 1 – Diagrama de casos de uso do sistema

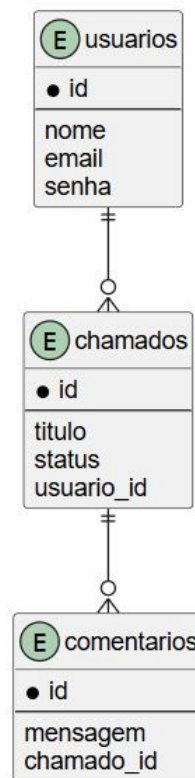


Fonte: Os autores (2025).

4.2 Modelo entidade-relacionamento

O modelo entidade-relacionamento, ilustrado de forma simplificada na Figura 2, evidencia os principais relacionamentos do domínio. Um usuário pode abrir diversos chamados (relação de um para muitos), e cada chamado pode receber diversos comentários. No esquema completo do banco de dados, esse núcleo é complementado pelas tabelas de categorias, prioridades, anexos, logs e notificações.

Figura 2 – Modelo entidade-relacionamento (núcleo)



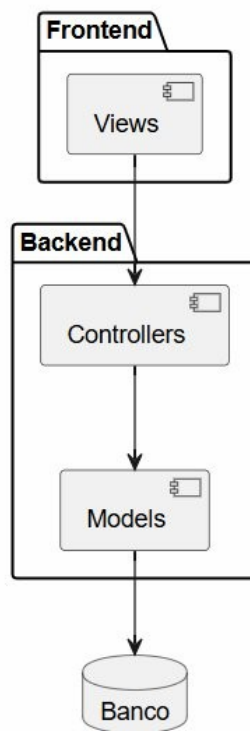
Fonte: Os autores (2025).

Além das entidades exibidas na figura, o banco de dados contempla as seguintes tabelas: usuarios (com perfil, departamento, CPF e data de nascimento), categorias (com ícone e cor), prioridades (com SLA em horas e nível), chamados (com status, atendente, prazo de SLA e avaliação), comentarios (com indicador de nota interna), anexos, logs (auditoria) e notificacoes. As relações utilizam chaves estrangeiras, com exclusão em cascata para comentários e anexos vinculados a um chamado.

4.3 Diagrama de componentes

O diagrama de componentes, exibido na Figura 3, representa a organização macro da aplicação segundo o padrão MVC. A camada de front-end concentra as visões (Views); a camada de back-end abriga os controladores (Controllers), que acionam os modelos (Models); e estes, por sua vez, comunicam-se com o banco de dados. O fluxo unidirecional reforça a separação de responsabilidades entre as camadas.

Figura 3 – Diagrama de componentes (arquitetura MVC)

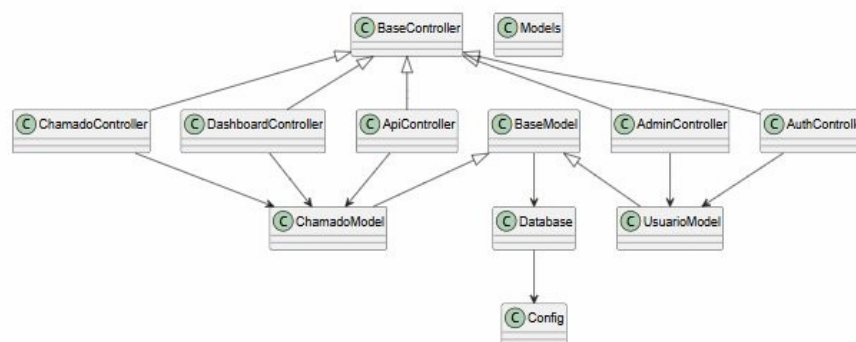


Fonte: Os autores (2025).

4.4 Diagrama de classes

O diagrama de classes, apresentado na Figura 4, detalha as principais classes do sistema e suas relações de herança e dependência. As classes `BaseController` e `BaseModel` são abstratas e centralizam comportamentos comuns: a primeira fornece renderização, redirecionamento, controle de sessão e proteção contra CSRF aos demais controladores; a segunda implementa as operações genéricas de CRUD para os modelos. A classe `Database`, responsável pela conexão via PDO, depende das configurações definidas em `Config`.

Figura 4 – Diagrama de classes do sistema



Fonte: Os autores (2025).

5 ARQUITETURA E IMPLEMENTAÇÃO

O HelpDesk Pro implementa o padrão MVC de forma artesanal, sem o uso de frameworks de aplicação. O ponto de entrada único é o arquivo `index.php`, que atua como front controller: ele interpreta os parâmetros da requisição, mapeia o controlador e a ação correspondentes e delega a execução. Esse desenho centraliza o roteamento e o carregamento das dependências.

5.1 Camada de modelo

A camada de modelo é estruturada a partir da classe abstrata `BaseModel`, que oferece os métodos genéricos `buscarPorId`, `criar`, `atualizar` e `deletar`, todos baseados em consultas preparadas. Os modelos específicos — como `UsuarioModel` e `ChamadoModel` — herdam essa classe e acrescentam consultas especializadas. O `ChamadoModel`, por exemplo, reúne a listagem de chamados com junções (joins) entre usuários, categorias e prioridades, o cálculo de indicadores de SLA vencido e estatísticas por status, categoria e período.

5.2 Camada de controle

Os controladores herdam a classe abstrata `BaseController`, que disponibiliza recursos transversais: renderização de visões, respostas em formato JSON, redirecionamentos, mensagens temporárias (flash), verificação de sessão e de perfil, além da geração e validação de tokens CSRF. Entre os controladores estão o `AuthController` (autenticação, registro e recuperação de senha), o `ChamadoController` (ciclo de vida dos chamados e Kanban), o `AdminController` (gestão de usuários, categorias e prioridades), o `DashboardController` (indicadores) e o `ApiController` (API REST).

5.3 Camada de visão

A camada de visão é composta por cerca de 23 arquivos PHP que combinam HTML e pequenos trechos de código para apresentação. As visões são organizadas por contexto — autenticação, chamados, painel, administração, área pública e páginas de erro — e reaproveitam cabeçalhos e rodapés por meio de arquivos parciais (partials). A apresentação utiliza Bootstrap 5 para garantir responsividade.

5.4 Segurança

O sistema adota diversas medidas de segurança. As senhas são armazenadas com a função `password_hash` e verificadas com `password_verify`, nunca em texto puro. Todo o acesso ao banco utiliza consultas preparadas, mitigando injeção de SQL. Os formulários são protegidos por tokens CSRF, gerados e validados a cada submissão. O acesso às áreas restritas é controlado por verificações de sessão e de perfil. O recurso de “lembrar-me” emprega um cookie assinado

com HMAC-SHA256, evitando adulteração. A recuperação de senha exige a confirmação combinada de e-mail, CPF e data de nascimento, com expiração temporal da etapa de redefinição.

5.5 API REST

O ApiController expõe um conjunto de endpoints que retornam dados em formato JSON, permitindo a integração com outras aplicações ou o consumo assíncrono pela própria interface. Estão disponíveis a listagem de chamados do usuário autenticado, o detalhamento de um chamado específico, estatísticas gerais (restritas a atendentes e administradores) e as notificações do usuário.

6 FUNCIONALIDADES

As principais funcionalidades implementadas no HelpDesk Pro são:

- Autenticação com login, logout, registro de novos clientes e recuperação de senha;
- Três perfis de acesso (cliente, atendente e administrador) com permissões distintas;
- Cadastro e acompanhamento completo de chamados (criação, edição, visualização e alteração de status);
- Classificação por categorias (com ícone e cor) e por prioridades com SLA configurável;
- Comentários públicos e notas internas, além de upload de anexos;
- Painel gerencial com indicadores e gráficos (Chart.js);
- Quadro Kanban com arrastar e soltar para gestão visual do fluxo;
- Alerta visual de SLA vencido e filtros avançados por status, categoria, prioridade e SLA;
- Histórico de alterações e auditoria de ações;
- Sistema de notificações e relatórios gerenciais;
- API REST em JSON para consulta de dados.

7 CONSIDERAÇÕES FINAIS

O HelpDesk Pro atingiu os objetivos propostos, entregando um sistema web funcional para a gestão de chamados corporativos. A aplicação do padrão MVC em PHP, aliada ao uso de classes abstratas para o reaproveitamento de comportamentos, resultou em um código organizado e de manutenção facilitada. As medidas de segurança adotadas — consultas preparadas, proteção CSRF e hash de senhas — alinham o projeto às boas práticas de desenvolvimento web.

O processo de modelagem, materializado nos diagramas de casos de uso, entidade-relacionamento, componentes e classes, mostrou-se fundamental para orientar a implementação

e assegurar a coerência entre o projeto e o produto final. Como trabalhos futuros, sugere-se a adoção de um mecanismo de notificações por e-mail, a inclusão de testes automatizados e a evolução da API para autenticação por tokens, ampliando as possibilidades de integração do sistema.

REFERÊNCIAS

BOOTSTRAP. Bootstrap: the most popular HTML, CSS, and JS library in the world. Disponível em: <https://getbootstrap.com>. Acesso em: 2025.

CHART.JS. Chart.js: simple yet flexible JavaScript charting library. Disponível em: <https://www.chartjs.org>. Acesso em: 2025.

MYSQL. MySQL 8.0 Reference Manual. Oracle Corporation. Disponível em: <https://dev.mysql.com/doc>. Acesso em: 2025.

PHP. PHP Manual. The PHP Group. Disponível em: <https://www.php.net/manual>. Acesso em: 2025.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, I. Engenharia de software. 10. ed. São Paulo: Pearson, 2018.